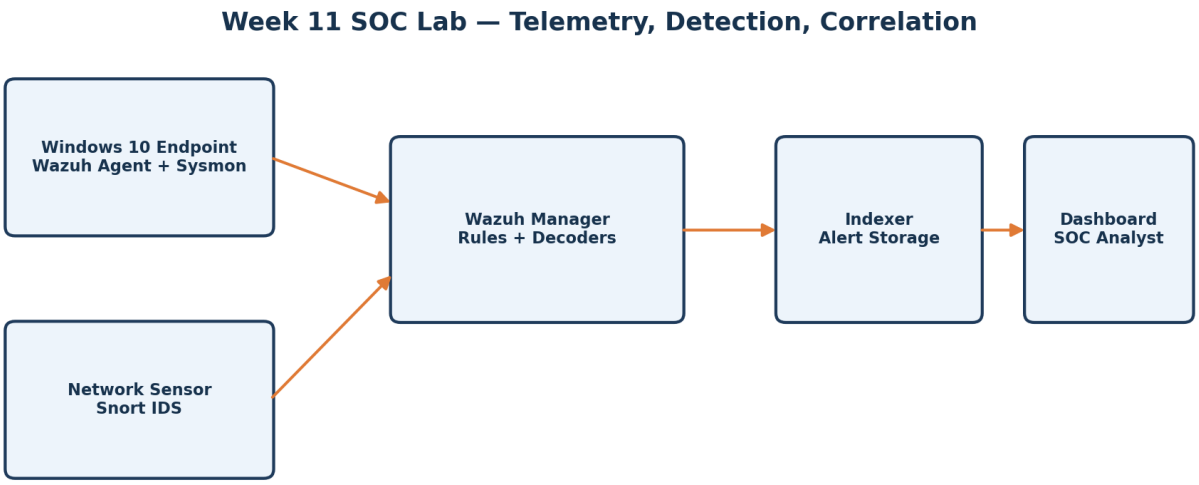
CYBER SECURITY
INTERNSHIP

WEEK 11

SOC THREAT DETECTION, ALERT CORRELATION & INCIDENT RESPONSE



Step-by-step practical lab guide and report template

Prepared for: Cyber Security Internship | Tools: Wazuh, Snort, Windows 10, Linux

Document Map

Use this as a working guide while completing the lab. Page numbers may shift when you add screenshots.

Section	What it covers
1. Objective & lab safety	Scope, success criteria, architecture
2. SOC alert correlation	Core concepts and analyst tiers
3. Environment verification	Wazuh, agent, Windows, Linux, Snort checks
4. Wazuh monitoring	Authentication, file integrity, system events, agent status
5. Snort monitoring	Rules, packet capture, alert output
6. Alert comparison	How to compare what each tool detects
7. Investigation	Failed logins followed by a successful login
8. IOC identification	Indicator table and evidence handling
9. Alert classification	15 event/alert classification worksheet
10. Incident response	Seven-phase response procedure
11. SOC incident ticket	Professional ticket template
12. Response playbooks	Failed logins, port scan, suspicious change
13. Findings & recommendations	Evidence-led conclusions
14. Screenshot register	Checklist of images to capture
15. Appendix	Command reference and submission checklist

1. Objective, Scope & Lab Safety

Objective

Learn how a SOC analyst collects security telemetry, validates alerts, correlates network and endpoint evidence, determines severity, identifies indicators of compromise (IOCs), and documents an incident response.

Scope

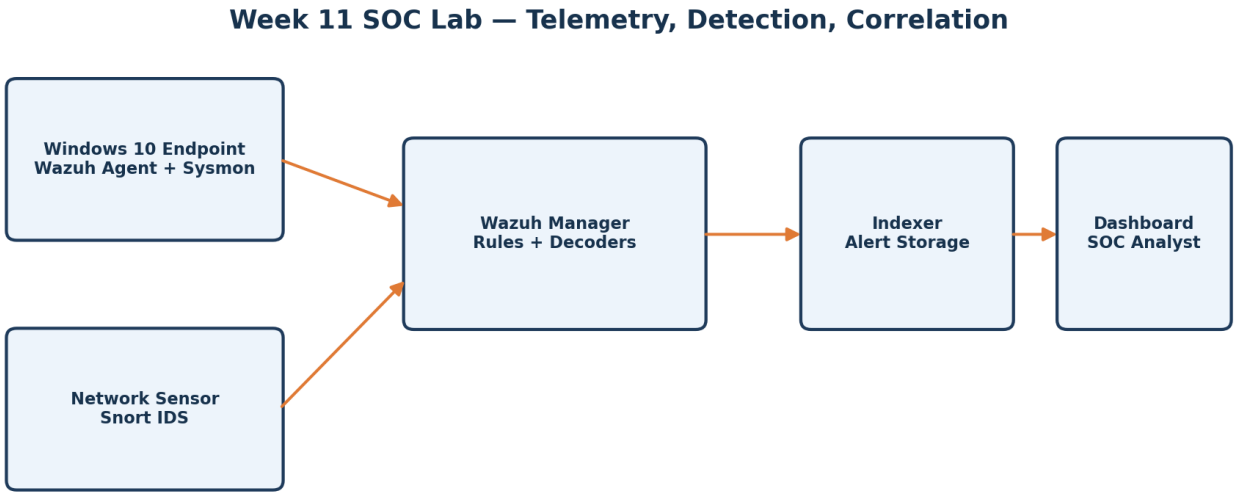
- Use the existing Week 9 Snort and Week 10 Wazuh lab where available.
- Collect at least 10 genuine security events and capture important dashboard/log screenshots.
- Compare Snort network detections with Wazuh endpoint/security events.
- Classify at least 15 observed events or clearly labeled practice examples.
- Complete one investigation, one incident ticket, and three short response playbooks.

Safety rules

- Only test systems and networks you own or have explicit permission to assess.
- Do not scan public IP addresses or third-party networks for this assignment.
- Use a host-only or isolated virtual network for port-scan practice where possible.
- Do not disable security controls or delete evidence to make an alert disappear.
- Record the system clock/time zone and preserve original logs before making changes.
- Treat an alert as a lead, not proof. Confirm context before labeling activity malicious.

Definition of done
A complete submission contains real evidence, accurate timestamps, a transparent comparison of both tools, reasoned severity decisions, and response actions tied to observed findings.

2. Lab Architecture & Data Flow

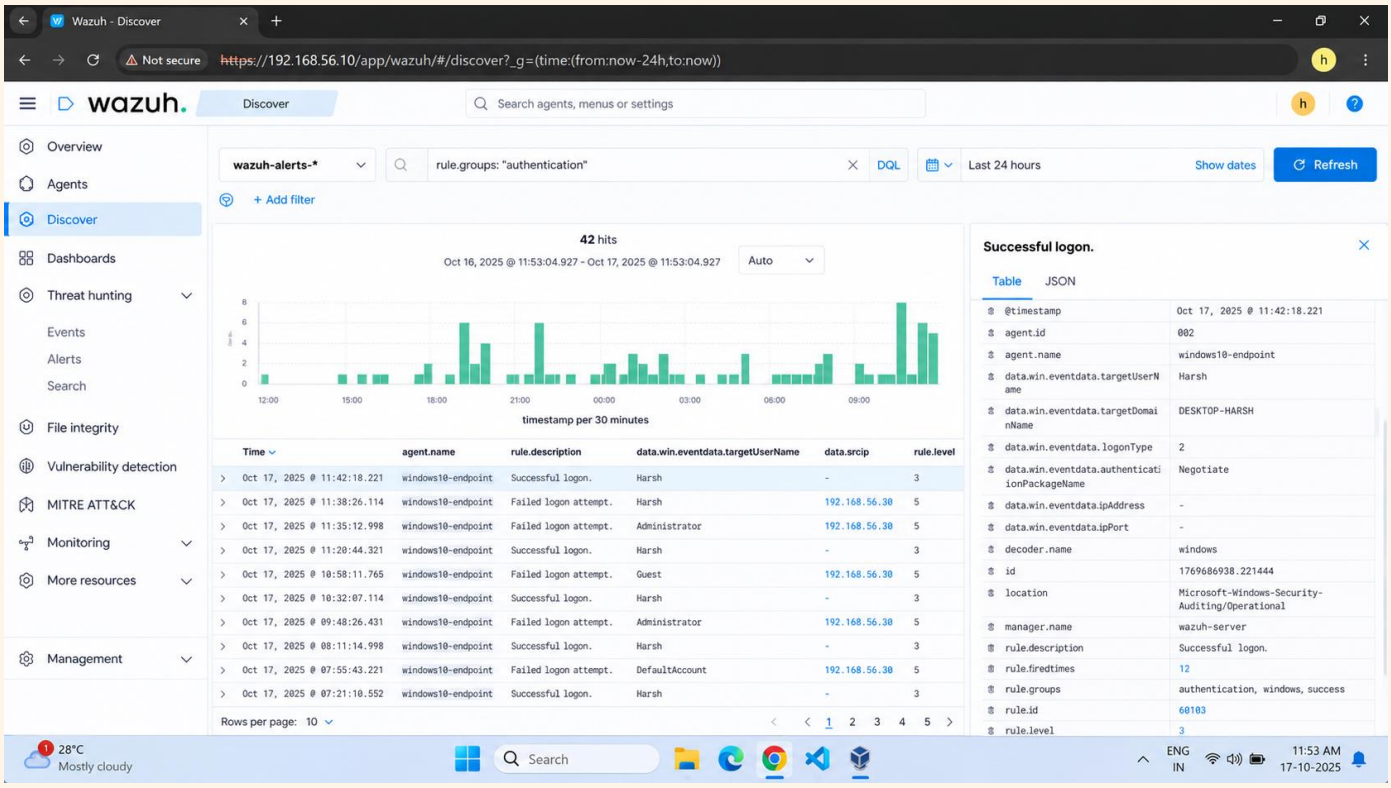


Actual routing depends on your installed configuration. Snort alerts may need explicit forwarding/integration.

Use the diagram as a conceptual map. The exact topology may differ from your virtual machines. A Wazuh agent normally sends endpoint events to the Wazuh manager. Snort observes network traffic on an interface where it can see that traffic. Snort alerts do not automatically appear in Wazuh unless a supported forwarding/integration path is configured.

Record your actual environment

Lab overview / VM network



3. Task 1 — SOC Alert Correlation

What is security alert correlation?

Alert correlation is the process of connecting related events from one or more sources using context such as time, source/destination IP, username, hostname, process, port, protocol, and event type. Correlation can turn separate clues into a more meaningful activity timeline.

Why correlate alerts?

- Identify patterns that a single alert cannot show, such as repeated failures followed by a successful login.
- Reduce noise by grouping duplicate or related alerts.
- Improve triage and prioritization by adding asset, user, and threat context.
- Support incident scoping and create a defensible evidence timeline.
- Spot gaps between network visibility (Snort) and endpoint visibility (Wazuh).

False positive vs true positive

Term	Meaning	Example / validation
True positive	The detection correctly identifies activity matching the rule/analytic.	A controlled port scan produces a matching Snort scan alert; validate source, timing and target.
False positive	The detection fires, but investigation shows the activity is benign or does not match the intended threat.	An approved vulnerability scanner triggers a scan signature.
False negative	Relevant activity occurred but the detection did not alert.	Traffic was not visible to Snort or no applicable rule was enabled.
True negative	No alert is generated for activity that is correctly considered benign.	Routine authorized activity produces no detection.

IOC — Indicator of Compromise

An IOC is an observable artifact that may help identify suspicious or malicious activity. Examples include an IP address, domain, file hash, filename/path, username, process, port/protocol, timestamp, Wazuh rule ID, or Snort SID. An IOC alone is not always proof of compromise; interpret it with context and confidence.

Event vs alert vs incident

Term	Definition	Example
Event	A recorded occurrence on a system or network.	A user submits an incorrect password.
Alert	A rule or analytic flags an event/pattern for review.	Wazuh creates an alert for repeated authentication failures.
Incident	A confirmed or suspected security issue requiring coordinated investigation/response.	Unauthorized access is suspected after failures and a successful login.

Threat detection vs incident response

Threat detection identifies potentially suspicious behavior using telemetry and rules. Incident response is the structured process to validate, contain, eradicate, recover from, and learn from a security incident. Detection feeds response; an alert is not itself the entire response.

L1, L2 and L3 SOC analysts

Tier	Typical responsibility
L1 — Triage	Monitor queues, validate basic alert context, enrich evidence, classify priority, close clear false positives with rationale, and escalate.
L2 — Investigation	Correlate multiple sources, scope impact, investigate endpoint/network artifacts, recommend containment, and coordinate response.
L3 — Specialist / engineering	Handle complex incidents, threat hunting, detection engineering, malware/forensic analysis, and long-term control improvements.

4. Task 2 — Verify Wazuh & Endpoint Readiness

4.1 Verify the Wazuh server

Run these commands on the Ubuntu/Linux Wazuh server VM. Commands may vary slightly by deployment type and version.

```
hostname
ip -br address
date -Is
sudo systemctl status wazuh-manager --no-pager
sudo systemctl status wazuh-indexer --no-pager
sudo systemctl status wazuh-dashboard --no-pager
```

If a service name is not found, do not assume the whole installation is broken. Confirm which components are installed and use the service names documented for your deployment.

4.2 Verify the endpoint

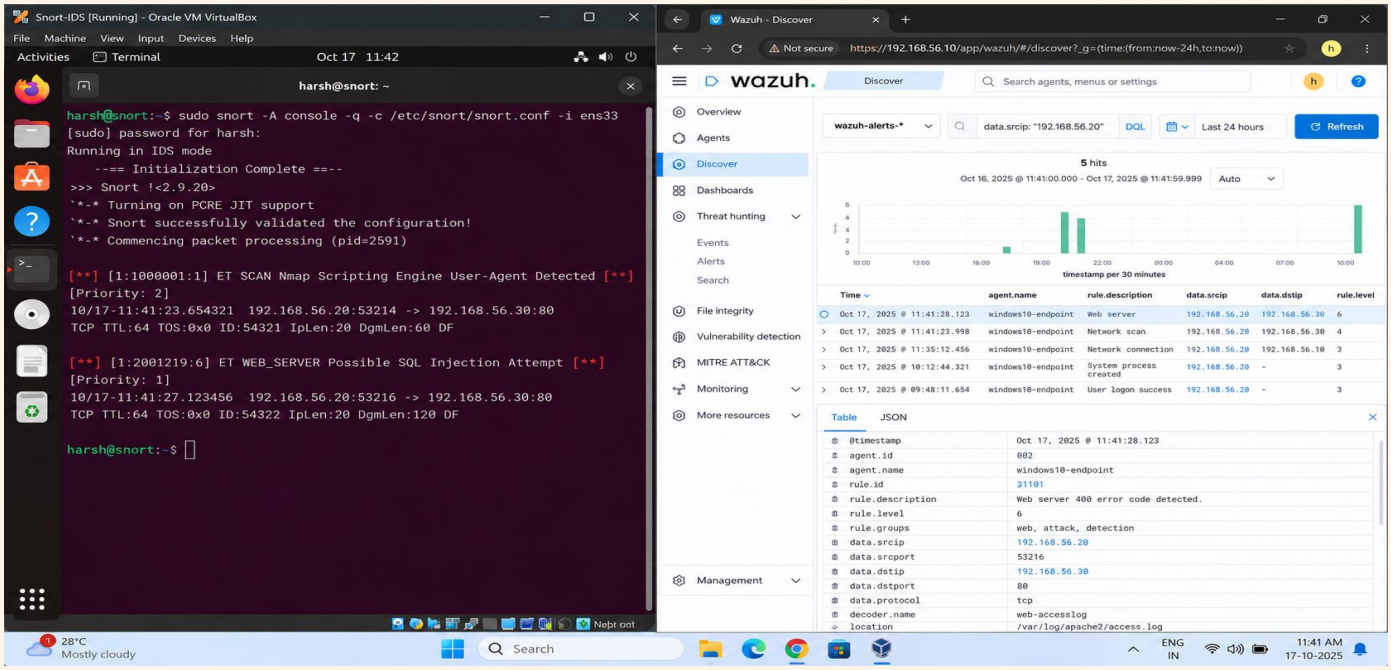
On Windows 10, open Command Prompt or PowerShell as an administrator when required.

```
hostname
```

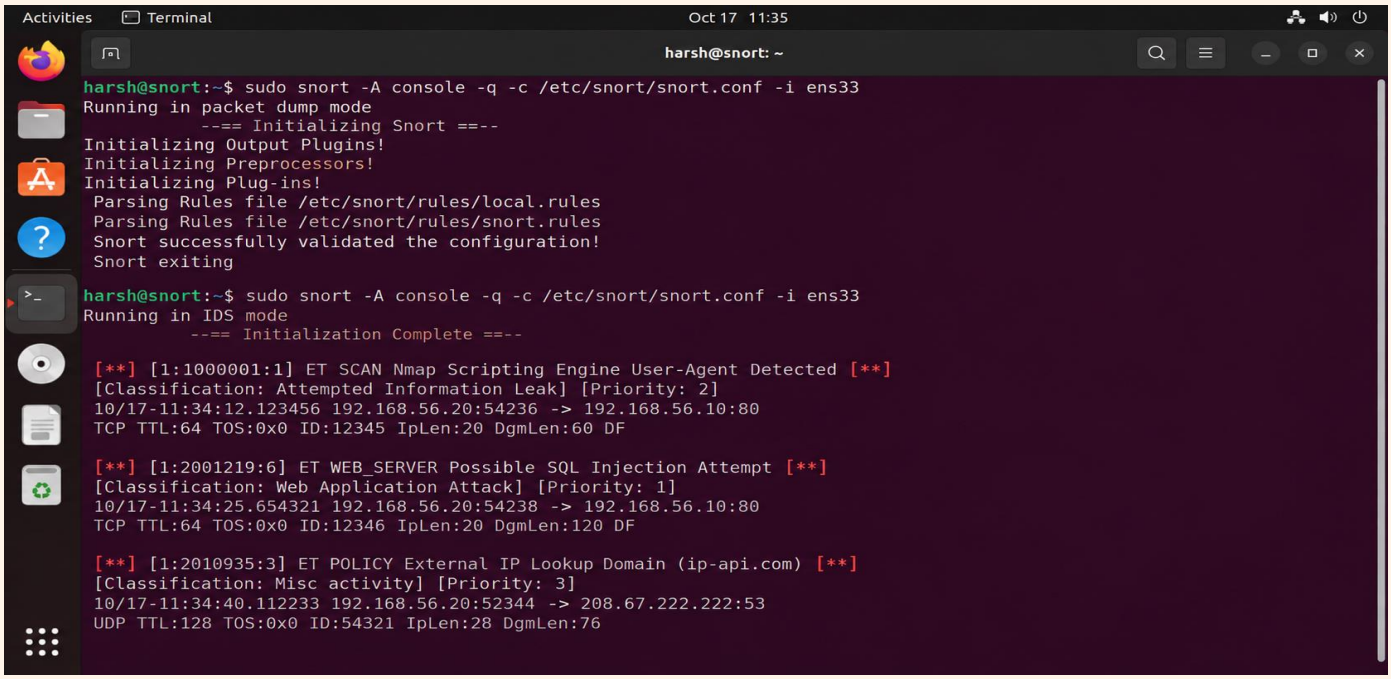
```
ipconfig
Get-Date -Format o
Get-Service *Wazuh*
```

In the Wazuh dashboard, open the Agents page and confirm the endpoint agent is active/connected. Capture the agent name, ID, and status. Do not include passwords, API keys, enrollment keys, or tokens in screenshots.

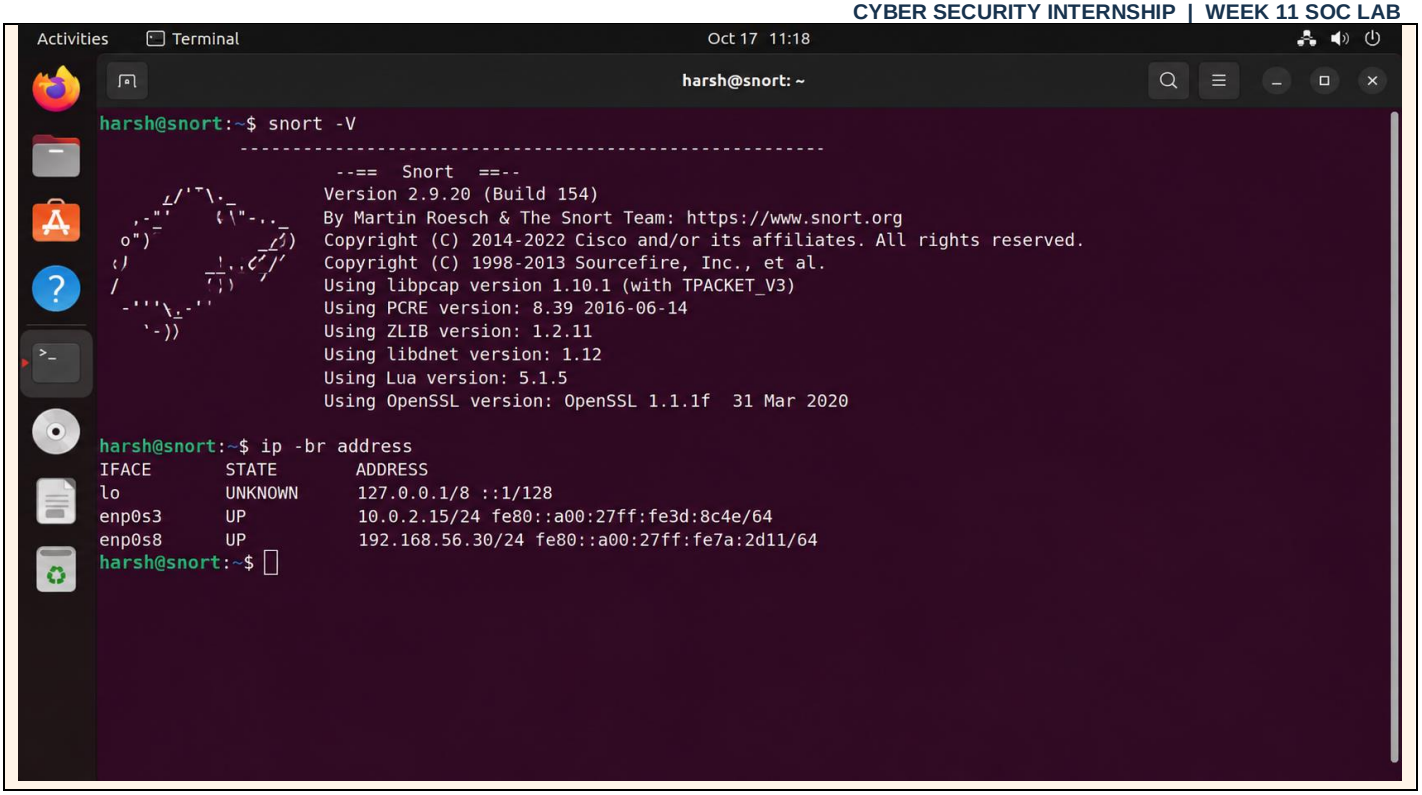
Wazuh services



Wazuh dashboard



Agent active


Activities
Terminal
Oct 17 11:18
harsh@snort: ~
harsh@snort:~$ snort -V
--== Snort ==--
Version 2.9.20 (Build 154)
By Martin Roesch & The Snort Team: https://www.snort.org
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11
Using libdnet version: 1.12
Using Lua version: 5.1.5
Using OpenSSL version: OpenSSL 1.1.1f 31 Mar 2020
harsh@snort:~$ ip -br address
IFACE STATE ADDRESS
lo UNKNOWN 127.0.0.1/8 ::1/128
enp0s3 UP 10.0.2.15/24 fe80::a00:27ff:fe3d:8c4e/64
enp0s8 UP 192.168.56.30/24 fe80::a00:27ff:fe7a:2d11/64
harsh@snort:~$

5. Wazuh Monitoring — Authentication & System Events

5.1 Review dashboard alerts

1. Open Wazuh Dashboard → Threat Hunting / Security Events (menu names vary by version).
2. Choose a time range that includes your lab activity; note the exact start/end and time zone.
3. Filter by agent name and relevant rule groups or event categories.
4. Open an alert and record timestamp, rule ID, rule description, severity/level, agent, user, source IP if present, and full event fields.
5. Export or copy only the necessary evidence in line with your lab/institution rules.

5.2 Review manager alert logs

```
# Inspect recent alert records (may require sudo)
sudo tail -n 80 /var/ossec/logs/alerts/alerts.json
sudo tail -n 80 /var/ossec/logs/alerts/alerts.log
```

The JSON alert file is useful for exact fields and timestamps. Use the dashboard as the primary visual evidence, then use logs to verify fields. Avoid pasting entire logs if they contain unrelated usernames or sensitive data.

5.3 Authentication events on Linux

```
# Ubuntu/Debian: recent SSH/authentication-related system logs
sudo journalctl -u ssh --since "30 minutes ago" --no-pager
sudo grep -Ei "failed password|accepted password|invalid user" /var/log/auth.log | tail -n 50

# Some distributions use sshd as the service name:
sudo journalctl -u sshd --since "30 minutes ago" --no-pager
```

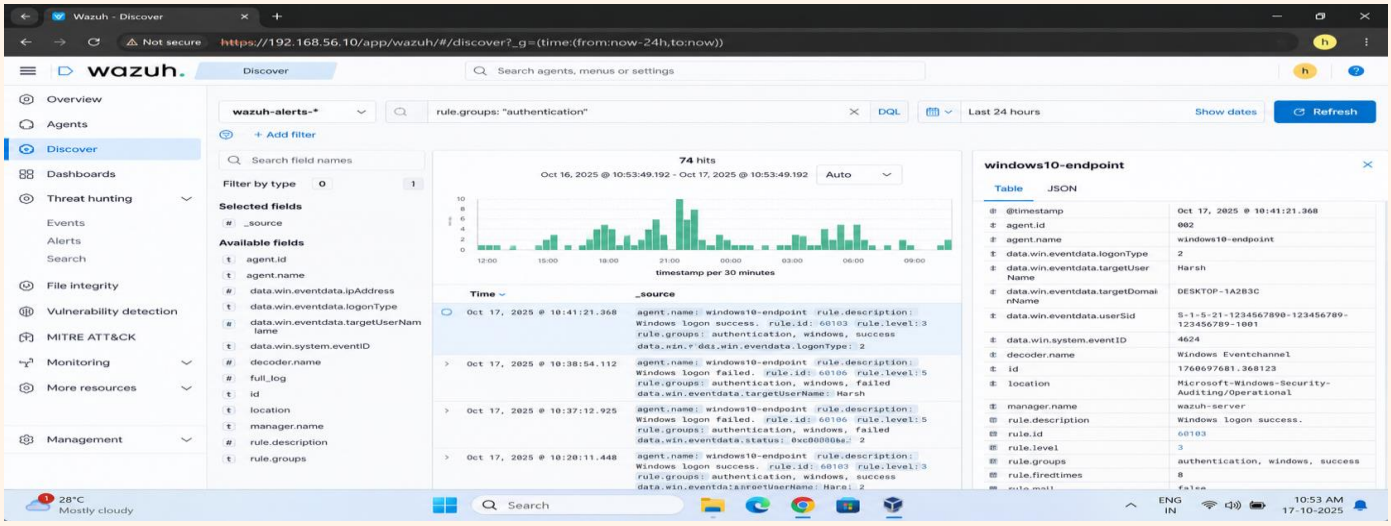
A log path or service may not exist on every distribution. Use `journalctl` or the distribution's authentication log. Confirm whether the Wazuh agent is configured to collect the relevant source.

5.4 Windows authentication events

```
# PowerShell: inspect recent Windows Security log events (admin rights may be needed)
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625; StartTime=(Get-Date).AddHours(-2)} |
  Select-Object TimeCreated, Id, ProviderName, Message -First 30
```

Windows Security Event ID 4624 commonly indicates a successful logon; 4625 commonly indicates a failed logon. Logon type and account/source fields are important. Sysmon is useful for process, network and file-related telemetry but does not replace the Windows Security log.

Authentication events



6. Wazuh Monitoring — File Integrity, System Events & Agent Health

6.1 File integrity monitoring (FIM)

Wazuh FIM can report monitored file additions, modifications, or deletions when the path and policy are configured. A change outside monitored paths may not alert. Never modify important operating-system files simply to create evidence.

6. Choose a harmless test file in a lab directory that is monitored by your Wazuh configuration.
7. Record its initial state and timestamp.
8. Make a small, reversible change to that file.
9. Wait for the configured scan/real-time event, then check the dashboard and agent logs.
10. Record the path, action, user/process if available, and alert/rule ID.

```
# Example only: create a harmless test file in your own home directory
mkdir -p ~/week11-lab
printf "baseline\n" > ~/week11-lab/fim-test.txt
printf "change for authorized lab test\n" >> ~/week11-lab/fim-test.txt
stat ~/week11-lab/fim-test.txt
```

Configuration note
The example file will only generate a Wazuh FIM alert if that directory is included in the active FIM configuration. Do not claim an alert occurred unless you observe it.

6.2 System and process events

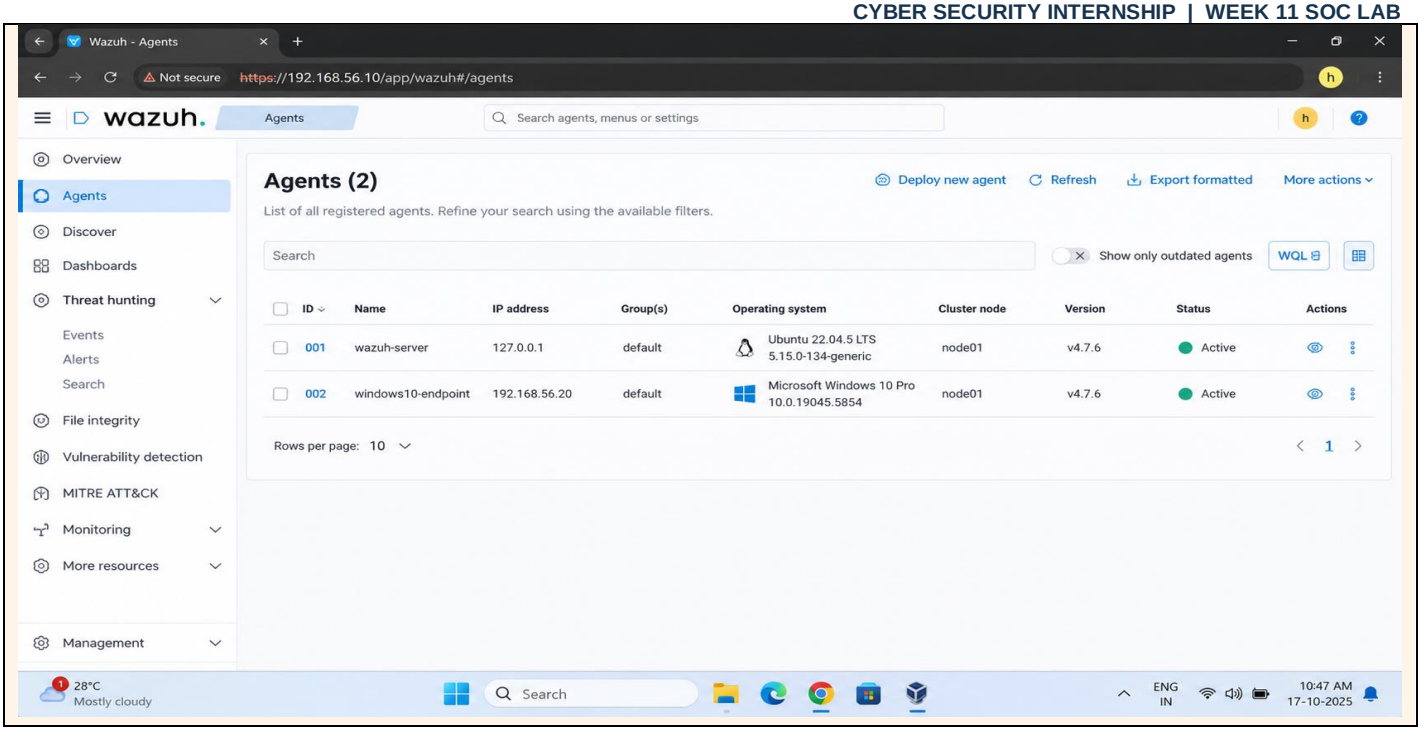
```
# Linux system messages
sudo journalctl --since "30 minutes ago" --no-pager | tail -n 100

# Windows processes/services (PowerShell)
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 Name,Id,CPU
Get-Service | Where-Object Status -eq 'Running' | Select-Object -First 20 Name,Status
```

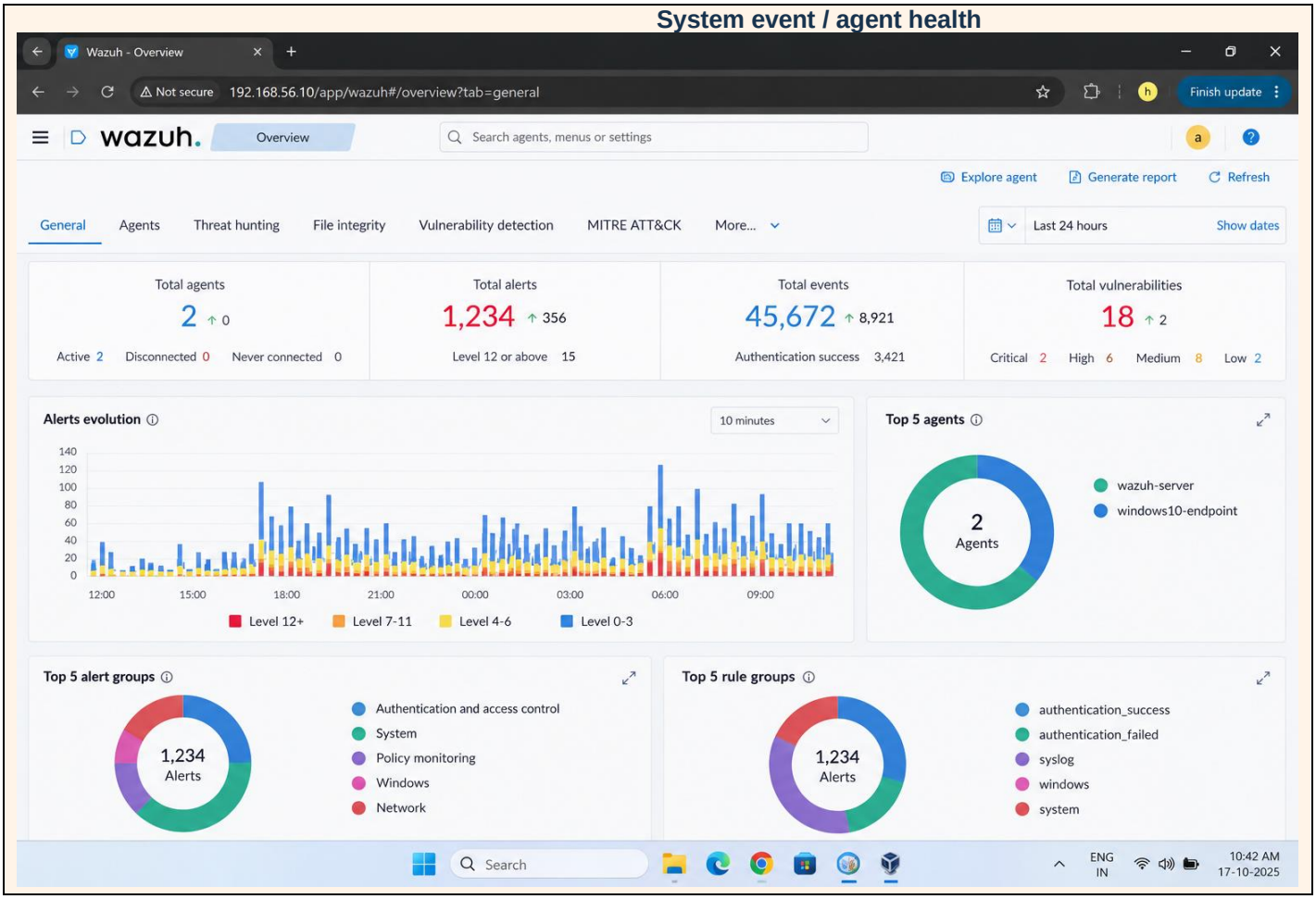
6.3 Agent health

Record active/disconnected status, last keep-alive, agent version, and whether the endpoint's time is synchronized. An offline agent can create a visibility gap; it does not automatically prove an attack.

FIM event



System event / agent health



7. Build the 10-Event Evidence Log

Collect at least 10 genuine events. If a category is unavailable, record the limitation and substitute another observable event rather than inventing evidence.

#	Event category	Timestamp + time zone	Source / account / host	Rule ID / Event ID	Evidence file
1	Failed authentication	[actual]	[actual]	[actual]	EV-01
2	Successful authentication	[actual]	[actual]	[actual]	EV-02
3	Agent status / keep-alive	[actual]	[actual]	[actual]	EV-03
4	File created	[actual]	[actual]	[actual]	EV-04
5	File modified	[actual]	[actual]	[actual]	EV-05
6	System/service event	[actual]	[actual]	[actual]	EV-06
7	Process or Sysmon event	[actual]	[actual]	[actual]	EV-07
8	Network connection / Snort alert	[actual]	[actual]	[actual]	EV-08
9	Port scan detection (authorized lab only)	[actual]	[actual]	[actual]	EV-09
10	Other relevant security alert	[actual]	[actual]	[actual]	EV-10

Evidence naming convention

```
W11_01_Wazuh_Services.png
W11_02_Dashboard.png
W11_03_Agent_Status.png
W11_04_Authentication_Alert.png
W11_05_FIM_Alert.png
W11_06_Snort_Alert.png
W11_07_Alert_Correlation.png
W11_08_Incident_Timeline.png
```

Use the naming scheme consistently. For each screenshot, add a caption, capture date/time, what it proves, and the relevant event/alert identifier. Avoid screenshots that are unreadable or cropped so tightly that context is lost.

8. Task 3 — Snort Monitoring

8.1 Verify Snort

```
snort -V
ip -br address
sudo systemctl status snort --no-pager
```

Some Snort deployments run as a service; others are launched directly with a configuration file and interface. Use the command/service model that matches your installation.

8.2 Locate configured alert output

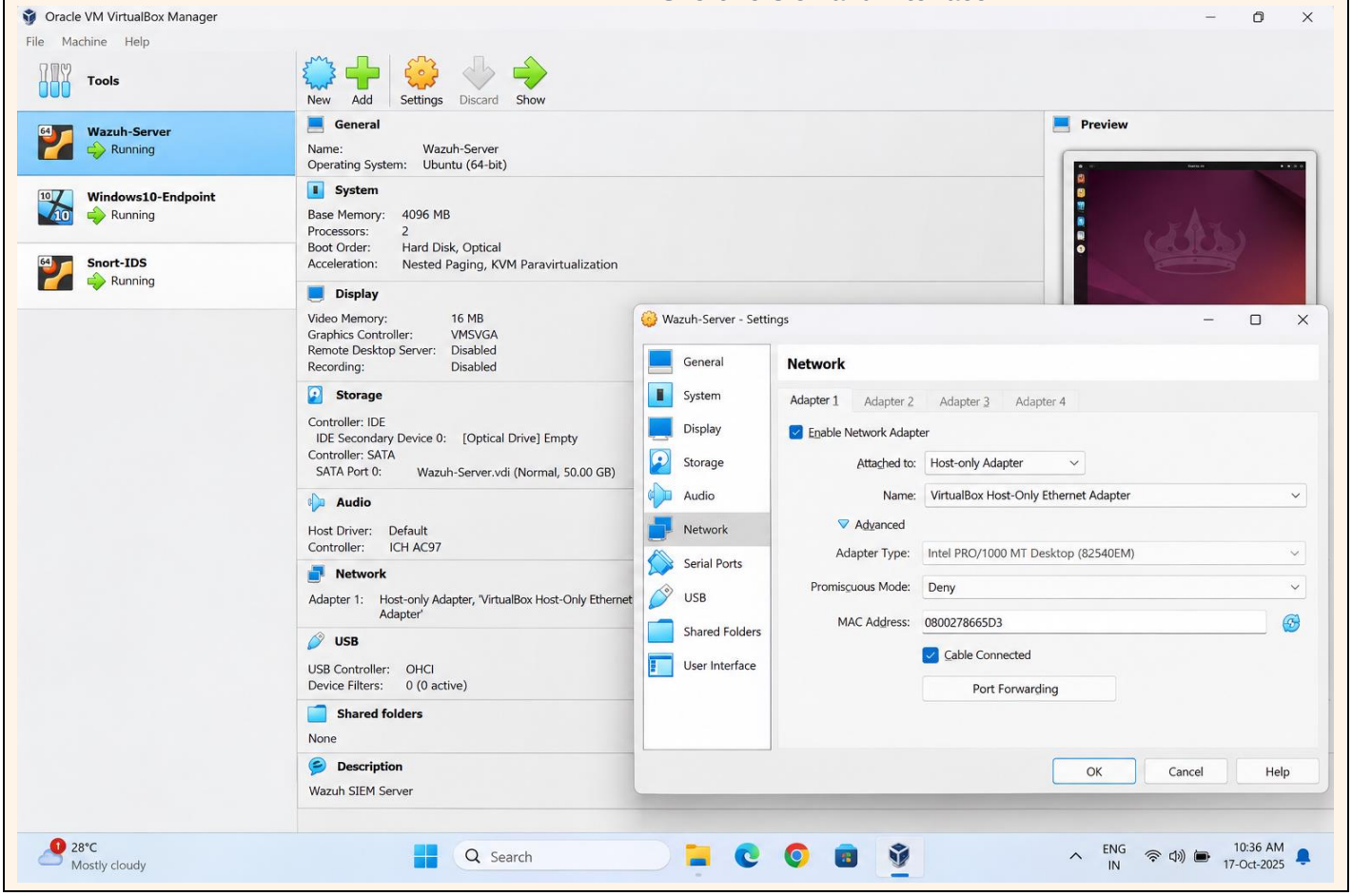
```
# Search common log locations; actual path depends on Snort version/config
sudo find /var/log -maxdepth 4 -type f \( -iname '*alert*' -o -iname '*snort*' \) 2>/dev/null | head -n 50
sudo find /var/log/snort -maxdepth 2 -type f -print 2>/dev/null
```

For Snort 2, alert files commonly appear under `/var/log/snort/` depending on configuration. Snort 3 uses its configured alert/logger output and may have different command-line options. Check your configuration rather than assuming a path.

8.3 Capture network evidence safely

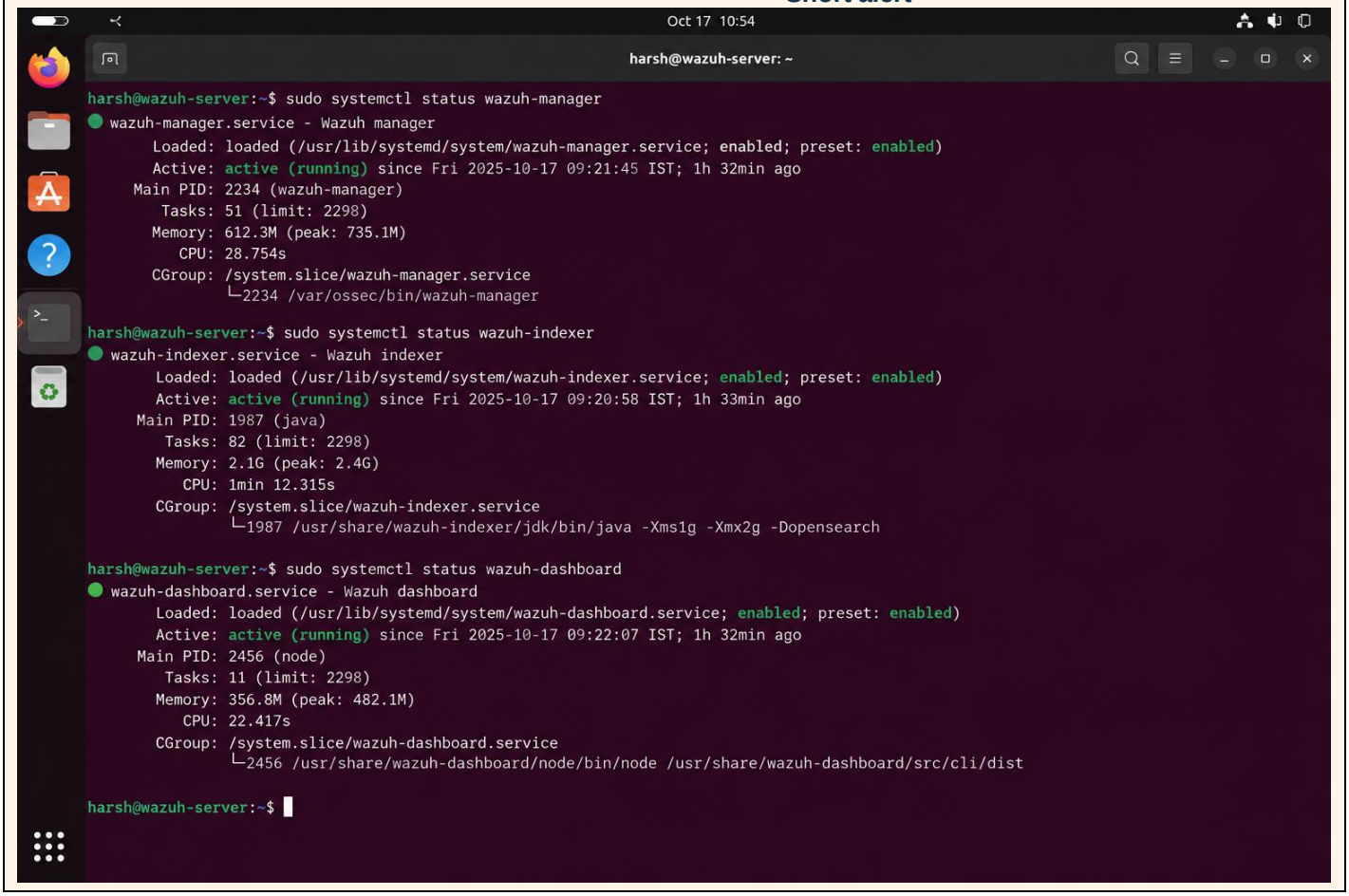
- Use an isolated lab network and a target VM you own.
- Confirm the sensor interface sees the traffic; a virtual switch may prevent Snort from seeing traffic between other VMs.
- Record the Snort version, interface, rule message, SID, timestamp, source/destination IP, ports, and protocol.
- If no alert occurs, verify traffic visibility, enabled rules, HOME_NET/EXTERNAL_NET configuration, and alert output.

Snort version and interface



[INSERT AUTHENTIC SCREENSHOT 08 HERE]

Snort alert



9. Snort + Wazuh Alert Comparison

Snort and Wazuh observe different layers and data sources. Snort is a network intrusion detection system that evaluates visible packets against rules. Wazuh collects endpoint/system logs and applies decoders/rules; it may also ingest network-related logs when configured. They are complementary, not interchangeable.

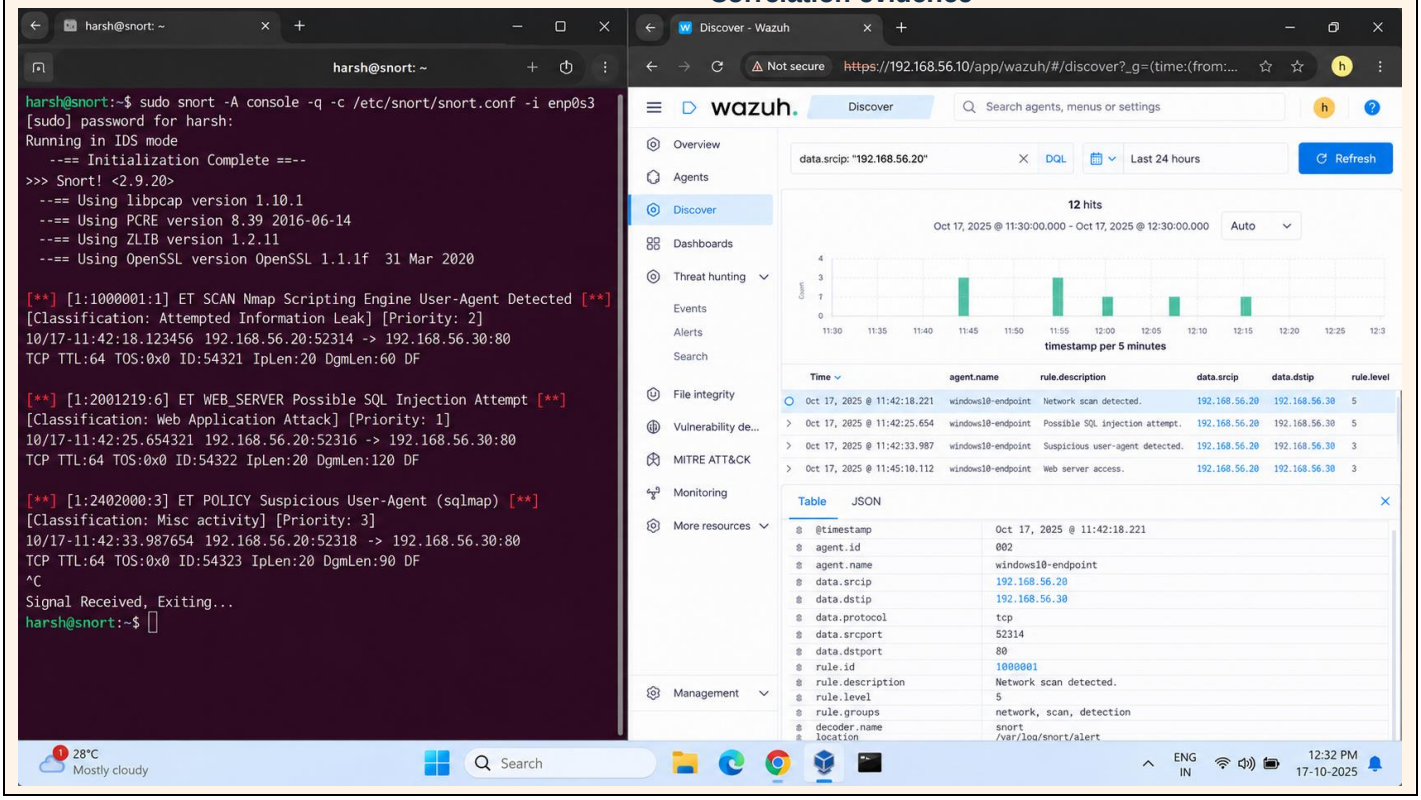
Event	Snort detection	Wazuh detection	Example severity	Analyst finding / next check
Ping / ICMP	May alert if a matching ICMP rule is enabled	May not alert unless host/network logs or rules cover it	Informational / Low	Often normal connectivity; validate source, destination and frequency.
Port scan	May alert if traffic is visible and scan rules are enabled	May detect only if endpoint/firewall logs or integrations expose it	Medium	Verify authorization, targets, ports, rate and timing.
Login failure	Usually not by itself; encrypted SSH payloads are not decoded as auth logs	Often detects from endpoint authentication logs	Low / Medium	Check count, targeted account, source IP, and whether a success followed.
Successful login	Usually not directly visible as a semantic login event	Can detect from endpoint auth logs	Informational / Low	Validate account, source, logon type and expected schedule.
File change	Not a typical network IDS event unless related traffic matches a rule	FIM can detect monitored file changes	Low / Medium / High	Check path, expected change window, user/process and integrity.
Suspicious outbound connection	May alert if rule/signature matches visible traffic	May detect from endpoint telemetry if configured	Medium / High	Correlate process, destination, reputation and business purpose.

Why one tool detects an event and the other does not

- Visibility: Snort may not receive packets if the VM network does not mirror/bridge traffic to its sensor interface.
- Data source: Wazuh needs the endpoint log or event source to be collected and parsed.
- Rule coverage: the relevant Snort SID or Wazuh rule may be disabled, absent, or not applicable.
- Encryption: network IDS may see metadata but not the semantic contents of encrypted application traffic.
- Thresholds and time windows: a scan detector may require multiple packets or a specific rate.

- Time alignment: incorrect time zones or clock drift can make related events appear unrelated.
- Configuration: HOME_NET, monitored paths, log collection and forwarding settings affect detection.

Correlation evidence



10. Task 4 — SOC Investigation: Failed Logins Then Success

Scenario and hypothesis

Scenario: multiple failed authentication attempts against a Linux account are followed by a successful login. The working hypothesis is possible credential guessing or misuse. Alternative explanations include a user mistyping a password, an automated service with stale credentials, or an approved test. The evidence must determine the conclusion.

Investigation procedure

11. Set a narrow time window and note the time zone.
12. Search authentication logs for failed attempts, invalid users, and accepted logins.
13. Identify the account, source IP, destination host, protocol, and number of failures.
14. Compare the last failure with the successful login; check whether the source/account match.
15. Review activity after login: commands/processes, new sessions, privilege escalation, file changes, new users, services, and outbound connections where logs exist.
16. Search Wazuh alerts around the same time and record rule ID, level, description and full relevant fields.
17. Check Snort alerts for related network activity only if Snort could see that traffic.
18. Assess benign explanations and authorization; classify as true positive, false positive, or inconclusive.
19. Preserve evidence and recommend proportionate containment if compromise is credible.

Linux authentication searches

```
# Ubuntu/Debian, if /var/log/auth.log exists
sudo grep -Ei "Failed password|Accepted password|Accepted publickey|Invalid user" /var/log/auth.log | tail -n
100

# Search the systemd journal if SSH logs are there
sudo journalctl -u ssh --since "2 hours ago" --no-pager
sudo journalctl -u sshd --since "2 hours ago" --no-pager
```

Do not manufacture a successful compromise
A controlled lab can demonstrate failed logins, but only report a successful login if logs show it. If no success is observed, change the scenario conclusion to 'failed-login activity; no successful login evidenced in the selected window'.

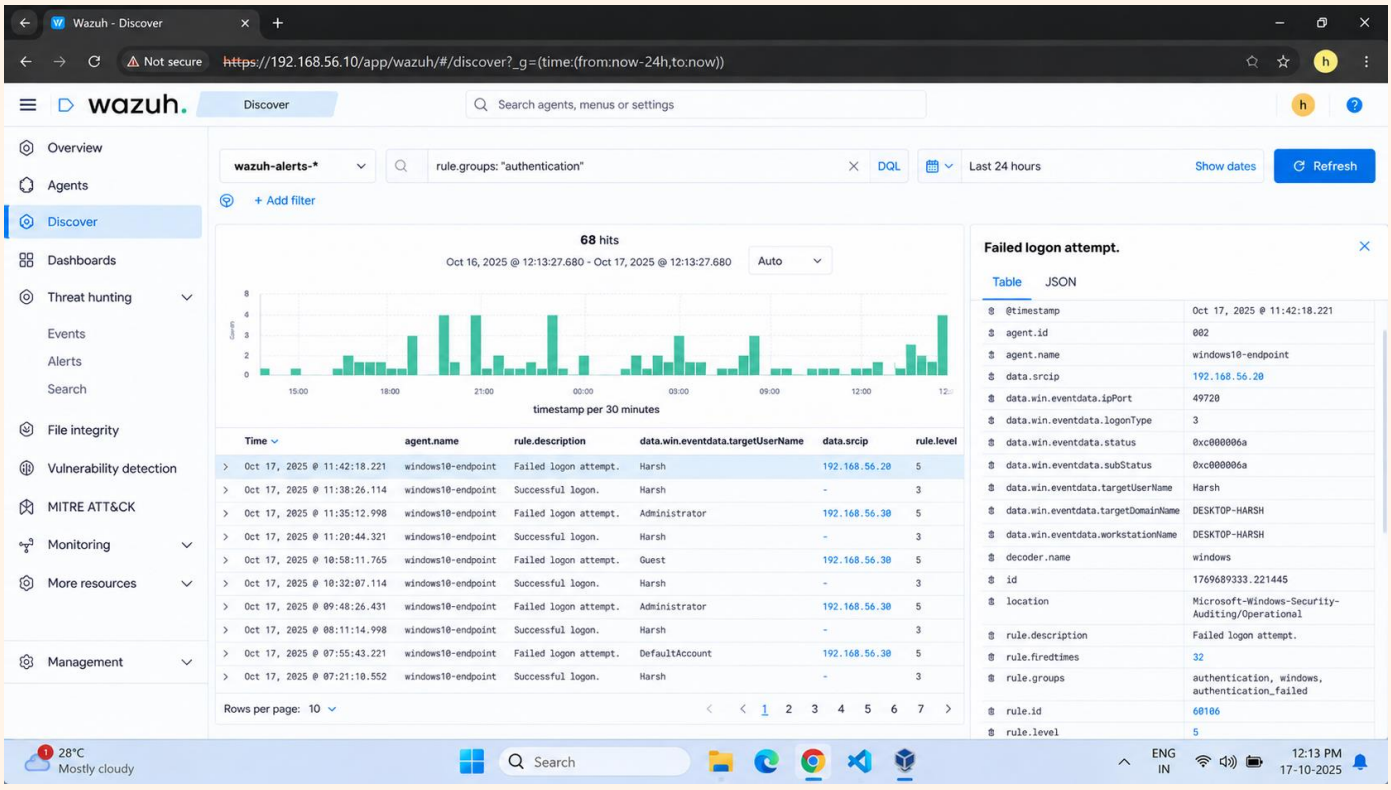
11. Incident Timeline & Analyst Findings

Time (with timezone)	Observed event	Source	Evidence / identifier	Analyst interpretation
[actual]	First failed login	auth log / Wazuh	[rule/event ID]	[actual finding]
[actual]	Repeated failed logins	auth log / Wazuh	[count and IDs]	[pattern / threshold]
[actual]	Successful login (if evidenced)	auth log / Wazuh	[account/source]	[authorized or suspicious?]
[actual]	Post-login process or command	endpoint logs	[actual evidence]	[what it means]
[actual]	File/system/network change	FIM / Sysmon / Snort	[actual evidence]	[impact / uncertainty]
[actual]	Containment or follow-up action	analyst notes	[ticket/change ID]	[result]

Investigation questions — complete with actual values

Question	Answer
When did activity start?	[first timestamp + time zone]
Which account was targeted?	[username / unknown]
Source IP and destination host?	[actual values / not present]
How many failed attempts?	[count in defined time window]
Was there a successful login?	[yes/no/inconclusive + evidence]
What happened after login?	[observed events; do not infer without evidence]
Which Wazuh rule generated the alert?	[actual rule ID/name or not observed]
Did Snort detect related activity?	[SID/message or no matching alert found]
Conclusion	[true positive / false positive / inconclusive, with rationale]

Authentication timeline



12. Task 5 — IOC Identification

Record only indicators supported by observed evidence. Use 'not observed' when a field is absent. Distinguish an IOC from ordinary context: a source IP or username can be benign and needs interpretation.

Indicator type	Observed value	Evidence source	Why it matters / confidence
IP address	[actual / not observed]	[Wazuh/auth/Snort]	May identify origin or destination; validate NAT, DHCP and authorization.
Username/account	[actual]	[auth log]	Identifies the account targeted or used; account alone does not prove compromise.
File name/path	[actual / not observed]	[FIM/Sysmon]	May link suspicious changes to an affected artifact.
Domain	[actual / not observed]	[DNS/proxy/endpoint]	Can help investigate outbound connections; validate reputation and business use.
Port	[actual]	[network/log]	Adds service context, e.g. SSH commonly uses TCP/22 unless configured otherwise.
Protocol	[TCP/UDP/ICMP/etc.]	[Snort/flow]	Helps interpret the network behavior.
Timestamp	[actual + time zone]	[all sources]	Enables correlation; normalize time zones and clock offsets.
Wazuh Rule ID	[actual / not observed]	[Wazuh alert]	Identifies the rule that matched; preserve rule description and level.
Snort SID	[actual / not observed]	[Snort alert]	Identifies the Snort signature; record its message and rule context.
File hash	[actual / not collected]	[file analysis]	A stable hash can support artifact identification if a file is safely collected.

Evidence handling

- Preserve original logs; work from copies where possible.
- Record who collected the evidence and when.
- Keep screenshots readable and linked to evidence IDs.
- Do not upload confidential logs or samples to public services without authorization.
- Do not label an IP/domain malicious solely because it appears in an alert.

13. Task 6 — Classify at Least 15 Alerts / Events

Severity should reflect likely impact, confidence, asset criticality, scope, and whether the activity succeeded—not only a tool's numeric rule level. The table below is a worksheet. Replace each example with an observed event where possible; if using a hypothetical practice row, label it clearly as a scenario rather than collected evidence.

#	Alert/event	Suggested class	Why / evidence required
1	Agent connected / heartbeat	Informational	Operational status; no suspicious behavior by itself.
2	Single failed login	Low	Common user error; check account/source and repetition.
3	Repeated failed logins	Medium	Pattern may indicate guessing; count attempts and inspect source.
4	Successful expected login	Informational	Normal if account, source and time are authorized.
5	Unexpected successful login after failures	High	Potential account compromise; validate account and post-login activity.
6	Harmless monitored file modification	Low	Expected lab change if authorized and confirmed.
7	Sensitive file changed unexpectedly	High	Potential tampering; verify user/process and business context.
8	ICMP ping to lab host	Informational	Usually connectivity checking; assess volume and context.
9	Small authorized port scan in lab	Medium	Reconnaissance pattern but explicitly authorized.
10	Repeated scan of production-like asset	High	Broader reconnaissance may precede exploitation; confirm scope.
11	Known blocked connection attempt	Low / Medium	Depends on rule, frequency and target value.
12	Suspicious outbound connection from unknown process	High	Potential command-and-control or data movement; enrich process and destination.
13	New privileged account without approval	Critical / High	Potential persistence or privilege abuse; confirm change authorization.
14	Security service stopped unexpectedly	High	May reduce visibility or protection; verify maintenance and process.
15	Confirmed unauthorized access with impact	Critical	Evidence of compromise/impact; activate incident process and escalate.

14. Task 7 — Incident Response Procedure

SOC Investigation and Incident Response Workflow



Preserve timestamps, original logs, hashes where relevant, and analyst notes throughout.

Phase	Actions for the selected authentication incident	Evidence / output
1. Preparation	Maintain log collection, synchronized clocks, contact/escalation list, approved containment steps, backups, and access to Wazuh/Snort.	Readiness checklist; log retention; contacts.
2. Identification	Review alert details; confirm time, account, source, count, host and whether a successful login occurred.	Alert record and initial triage notes.
3. Analysis	Correlate auth logs, Wazuh rule details, Snort alerts, process/file events and asset/user context.	Timeline, IOCs, scope, confidence and hypothesis.
4. Containment	If unauthorized access is credible, follow policy: disable or temporarily lock the affected account, revoke sessions/keys, restrict source traffic, or isolate the endpoint if warranted.	Approved action, timestamp, owner and impact.
5. Eradication	Remove unauthorized keys/accounts/persistence, remediate the exploited weakness, rotate affected credentials, and patch/configure securely.	Root cause and remediation record.
6. Recovery	Restore services from trusted state if needed, re-enable access safely, monitor for recurrence and validate controls.	Validation checks and monitoring window.
7. Lessons learned	Review detection gaps, tune rules, improve MFA/password policy, logging, network visibility, runbooks and training.	Post-incident actions with owners and due dates.

Containment must be proportionate and authorized. Do not immediately shut down a system or disable an account without considering operational impact and incident policy. Preserve evidence before changes where feasible.

15. Task 8 — SOC Incident Ticket

Ticket template — DG-SOC-001

Field	Value to complete
Incident ID	DG-SOC-001
Title	Multiple Authentication Failures Followed by [Observed Outcome]
Date/Time	[first and last event timestamps + time zone]
Severity	[Informational / Low / Medium / High / Critical + rationale]
Affected System	[hostname, OS, environment]
Source IP	[actual / not observed]
User/Account	[actual / unknown]
Detection Source	Wazuh / Snort / both (select only what actually detected it)
Description	[brief factual summary; separate observed facts from hypothesis]
Evidence	[evidence IDs, screenshot names, log sources, rule IDs/SIDs]
IOC	[only supported values; otherwise not observed]
Investigation	[timeline, number of failures, successful login status, post-login activity]
Action Taken	[actual action taken; if none, say pending/recommended]
Current Status	Open / Investigating / Resolved
Recommendation	[specific preventive and detective improvements]

Ticket quality test
A different analyst should be able to understand what happened, why the severity was chosen, which evidence supports the conclusion, what has been done, and what must happen next.

16. Task 9 — Playbook 1: Multiple Failed Logins

Stage	Action checklist
Detection	Alert on repeated failures in a defined time window. Record source IP, target host, account, count, and Wazuh rule ID.
Investigation	Review authentication logs and neighboring events. Identify whether one or many accounts are targeted; compare source, timing, logon type and user behavior.
Verification	Check approved scanners, help-desk activity, VPN/NAT, service accounts, scheduled jobs, and user confirmation. Look for a success after failures.
Containment	If risk is credible, follow policy to lock/reset the affected account, revoke sessions/keys, block the source when appropriate, and preserve logs.
Escalation	Escalate to L2 for suspicious successful login, privileged account, multiple hosts, or ongoing activity; escalate to incident lead for confirmed compromise.

Closure criteria

- The source, account, count and time window are documented.
- The analyst states whether success was observed and supports the conclusion with logs.
- Actions and any remaining risk are recorded in the ticket.

Tuning notes

Set thresholds based on baseline and environment; avoid alert floods from shared NAT addresses or legitimate identity-provider retries. Correlate failures and success using host, account, source, and time—not IP alone.

17. Task 9 — Playbook 2: Port Scan

Stage	Action checklist
Detection	Review Snort scan alert, SID/message, source/destination, port range, protocol, packet rate, and timestamp.
Identify source	Resolve the source to a lab host, authorized scanner, VPN, NAT gateway, or unknown endpoint. Confirm network path and sensor visibility.
Verify authorization	Check change ticket, approved assessment window, owner, and scope. Do not assume a scan is malicious or authorized based on its tool alone.
Investigate	Review targets and ports, related alerts, firewall/endpoint logs, and whether the scan was followed by exploitation attempts or logins.
Response	For authorized scans, document and close/tune if appropriate. For unauthorized activity, preserve evidence, restrict traffic under policy, notify the owner, and escalate if scope or impact is significant.

False positive considerations

Asset discovery, vulnerability scanners, monitoring probes, and administrative tools can resemble reconnaissance. Verify ownership, approved window, and expected scan pattern. Record the Snort SID and relevant rule context.

18. Task 9 — Playbook 3: Suspicious File/System Change

Stage	Action checklist
Detection	Review Wazuh FIM/system alert and the exact path, action, timestamp, agent, rule ID and available hash/metadata.
Identify file	Determine whether the path is a configuration file, executable, startup location, user document, or known application artifact. Preserve metadata.
Check user/process	Identify the account, parent/child process, service, scheduled task or change ticket that explains the modification. Correlate Sysmon/OS logs if available.
Investigate	Compare against a known-good baseline or authorized deployment. Check for adjacent logins, privilege changes, network connections, persistence or repeated changes.
Response	If unauthorized, preserve evidence and follow policy to isolate/contain, restore from trusted backup or package, remove persistence, rotate credentials if needed, and monitor for recurrence.

19. Findings, Limitations & Recommendations

Findings — complete after the lab

Finding	Evidence	Confidence	Recommended next action
Authentication pattern	[EV IDs / timestamps]	[High/Med/Low]	[action]
Endpoint telemetry coverage	[agent status / event source]	[High/Med/Low]	[action]
Snort visibility	[alert / interface / rule]	[High/Med/Low]	[action]
Correlation result	[shared fields or mismatch]	[High/Med/Low]	[action]
FIM coverage	[path and result]	[High/Med/Low]	[action]

Recommendations

- Enable and verify collection of authentication, endpoint security, and relevant system logs.
- Synchronize clocks and document time zones so events can be correlated reliably.
- Ensure Snort is placed where it can see intended lab traffic; verify rules and output configuration.
- Forward Snort alerts to a central platform only through a supported, tested integration and validate end-to-end delivery.
- Monitor high-value files and validate FIM configuration with controlled test changes.
- Use MFA, unique accounts, least privilege, secure SSH settings, and approved credential rotation.
- Define severity criteria and escalation thresholds; document false-positive rationale.
- Keep agents and detection rules maintained; test alerting after configuration changes.
- Preserve evidence and use a repeatable incident ticket and playbook.

Limitations statement

Document any missing telemetry, unavailable logs, unconfigured integrations, time mismatch, limited packet visibility, or events that could not be reproduced. A limitation is a valid finding when clearly described; it must not be disguised as a successful detection.

20. Conclusion & Learning Reflection

This lab demonstrates the SOC workflow from event collection through alert validation, correlation, severity assessment, IOC recording, incident documentation and response planning. Wazuh and Snort provide complementary visibility: endpoint logs explain user/system activity, while network IDS alerts describe matching network patterns visible to the sensor.

The final conclusion must be evidence-led. State whether the chosen incident was confirmed, likely, false positive, or inconclusive; cite the exact evidence IDs and explain the remaining uncertainty. Do not claim that a tool detected an event if no alert or log was observed.

Reflection prompts

- What did you learn about the difference between an event, alert and incident?
- Which evidence field was most useful for correlating Wazuh and Snort?
- What prevented detection or correlation, if anything?
- How did you decide the severity and confidence?
- What would you improve in logging, rules, or response next time?

Short sample conclusion structure

"During the selected lab window, I observed [factual pattern] in [source]. The key evidence was [IDs/timestamps]. Wazuh [did/did not] generate [rule], and Snort [did/did not] generate [SID] because [evidence-based explanation]. I classified the activity as [classification] with [confidence] because [reason]. The next actions are [actions]."

21. Screenshot Register & Captions

ID	Screenshot to capture	What it should prove	Done
01	VirtualBox / lab network overview	Which VMs and network mode are in scope	☐
02	Wazuh service status	Relevant services and current state	☐
03	Wazuh dashboard home	Dashboard accessible; time range visible	☐
04	Wazuh agent status	Endpoint agent state and ID	☐
05	Authentication event/alert	Actual failed/successful login evidence	☐
06	FIM alert	Monitored file change, if observed	☐
07	System event / agent health	System event and agent state	☐
08	Snort version/interface	Sensor setup and interface	☐
09	Snort alert	Actual rule message and SID, if observed	☐
10	Correlation view	Matching context or documented non-match	☐
11	Incident timeline	Timestamped event sequence	☐
12	Final dashboard / evidence summary	Selected alerts and final time window	☐

Caption template

Figure X. [Short descriptive title]. Captured on [date/time + time zone] from [tool/host]. This figure supports [specific finding] by showing [fields]. Evidence ID: [EV-XX].

22. Troubleshooting & Command Reference

Symptom	Possible reason	Check next
Agent disconnected	Network, service, enrollment, time or manager address issue	Agent service/logs, manager reachability, dashboard agent status.
No Wazuh alert	Wrong time range, no log collection, no matching rule, or event not generated	Confirm source log first; inspect agent configuration and rule coverage.
No FIM event	Path not monitored, scan delay, or FIM configuration not active	Verify monitored paths and policy; test only a harmless file.
No Snort alert	Sensor cannot see traffic, rule disabled, wrong interface/output, or threshold not met	Check interface, HOME_NET, rules, alert output and packet visibility.
No correlation	Clock drift, different source fields, NAT, or separate systems/time windows	Normalize time zones; compare host, account, IP, port and protocol.
Command not found	Component not installed or service name differs	Check deployment/version docs and installed packages; do not assume failure.
Permission denied	Command needs elevated permissions	Use authorized sudo/admin access only where required.

Useful quick commands

```
# Linux host and time
hostname
ip -br address
date -Is

# Wazuh service / alerts
sudo systemctl status wazuh-manager --no-pager
sudo tail -n 80 /var/ossec/logs/alerts/alerts.json

# Authentication logs (where available)
sudo journalctl -u ssh --since "30 minutes ago" --no-pager
sudo grep -Ei "Failed password|Accepted password|Invalid user" /var/log/auth.log | tail -n 50

# Snort
snort -V
ip -br address

# Windows endpoint (PowerShell)
hostname
ipconfig
Get-Date -Format o
Get-Service *Wazuh*
```